SN Computer Science (2025) 6:522
https://doi.org/10.1007/s42979-025-04008-y

ORIGINAL RESEARCH

A Benchmark for Physics-informed Machine Learning of Chlorine Concentration States in Water Distribution Networks

Luca Hermes[1] · André Artelt[1,2] · Stelios G. Vrachimis[2] · Marios M. Polycarpou[2,3] · Barbara Hammer[1]

Received: 22 November 2024 / Accepted: 28 April 2025
© The Author(s) 2025

Abstract
Ensuring high-quality drinking water is a critical responsibility of water utilities, with chlorine being the main disinfectant typically used. Accurate estimation of chlorine concentrations in the dynamic environment of water distribution networks (WDNs) is essential to ensure safe water supply. This work introduces a comprehensive and carefully created benchmark for training and evaluation of chlorine concentration estimation methodologies in WDNs. The benchmark includes a diverse dataset of 18,000 scenarios of the widely studied 'Hanoi', 'Net1', and the more recent and complex 'CY-DBP' water networks, featuring various chlorine injection patterns to capture diverse physical dynamics. To provide baseline evaluations, we propose and evaluate two neural surrogate models for chlorine state estimation: a physics-informed Graph Neural Network (GNN) and a physics-guided Recurrent Neural Network (RNN).

Keywords Water distribution networks · Water quality · Chlorine state estimation · Benchmark · Surrogate models · Deep learning · Graph Neural Networks · Recurrent Neural Networks

Introduction

Water distribution networks (WDNs) are critical infrastructures that deliver treated water to consumers across vast, interconnected networks. Ensuring water quality in WDNs is essential for public health and safety, as contaminants can arise from natural sources, accidental pollution, or malicious actions, posing significant risks to human health. Monitoring water quality across these networks is challenging due to their large size and the various factors that influence water quality as it travels through the system [1]. Traditional water quality monitoring methods depend on a few fixed sensors strategically located within the network to detect changes in parameters such as chlorine residuals, pH, and total organic carbon (TOC). These sensors serve as early warning indicators of contamination; however, their limited coverage often leaves parts of the network unmonitored, potentially allowing contamination events to go undetected [2].

In recent years, advances in machine learning (ML) have enabled the development of data-driven methods for water quality estimation in WDNs. These approaches leverage multivariate data from sensor networks and predictive modeling techniques to improve real-time monitoring, event detection, and anomaly identification, even in parts of the network lacking direct sensor coverage [3]. Early work by Perelman et al. [4] demonstrated how artificial neural networks (ANNs) combined with sequential probability updates can effectively detect contamination events by analyzing multivariate time-series data from sensors. Following this, models using hybrid methods, such as combining ANNs with support vector machines (SVMs) and Bayesian updating, have proven effective for real-time anomaly detection by processing high-dimensional data and dynamically updating the probability of an event [5].

Chlorine is a widely used disinfectant by water utilities worldwide to prevent the growth of harmful bacteria in treated water as it moves through distribution networks.

Luca Hermes and André Artelt have contributed equally to this work.

✉ André Artelt
aartelt@techfak.uni-bielefeld.de

[1] Bielefeld University, Inspiration 1, 33615 Bielefeld, NRW, Germany

[2] KIOS Research and Innovation Center of Excellence, University of Cyprus, Panepistimiou 1, Aglantzia, 2109 Nicosia, Cyprus

[3] Electrical and Computer Engineering Department, University of Cyprus, Panepistimiou 1, Aglantzia, 2109 Nicosia, Cyprus

Published online: 04 June 2025

When maintained at prescribed concentrations, it ensures public health safety while effectively inhibiting microbial growth. Accurately estimating chlorine concentrations across the entire network, including locations without direct sensor coverage, is challenging due to the limited number of sensors and significant modeling uncertainties [6]. This task, known as dense state estimation, aims to predict chlorine residuals throughout the WDN based on limited sensor data and has thus emerged as a key area of research. Li et al. [7] used a gated graph neural network (GGNN) to perform spatio-temporal dense state estimation, predicting chlorine concentrations across a real-world WDN in China. By incorporating historical data, sensor readings, and topological information, their model achieved high predictive accuracy for chlorine levels at unmonitored locations. Salem et al. [8] developed a Topology Adaptive Graph Neural Network (TAGNN) for purely spatial chlorine estimation, predicting chlorine concentrations across the network based on sparse sensor data at the current time step. Evaluated on the C-Town benchmark [9], the TAGNN model effectively used network structure, although it did not include temporal dynamics or flow directionality.

Other approaches have focused on specific node estimation, predicting chlorine residuals at critical nodes based on temporal data. Riyadh et al. [10] investigated convolutional neural networks (CNNs) and long short-term memory networks (LSTMs) to predict chlorine residuals in the main nodes of the network. Using a real-world dataset from a WDN in Canada, they demonstrated how temporal modeling can effectively capture the non-linear, dynamic relationships among operational parameters and chlorine levels, achieving high accuracy.

Moreover, due to the effect that certain contaminants have on chlorine concentration, several methods have indirectly used the estimation of chlorine concentration for contamination detection. Li et al. [11] explored contamination detection using generative adversarial networks (GANs), leveraging spatial-temporal correlations across multiple sensor locations to detect anomalies. Tested on real-world and synthetic data, this method improved detection capabilities for a variety of contamination events, highlighting the advantages of combining spatial and temporal modeling in WDNs. Similarly, Tinelli and Juran [2] applied SVMs and ANNs to monitor multiple water quality parameters for biocontamination, achieving robust results for early contamination detection in real-time scenarios.

However, the aforementioned approaches are not compared to other methods from the literature which is likely due to the fact that they address slightly different tasks and work on proprietary datasets. Consequently, comparing existing research on chlorine state estimation poses significant challenges due to the diverse datasets employed, encompassing both simulated and real-world data. These datasets are frequently not made publicly accessible, which hinders the ability of researchers to validate and replicate findings or to build upon previous work. The lack of standardized datasets and open access further complicates efforts to establish consistent evaluation metrics or benchmarks, leading to difficulties in assessing the relative performance of different methods and models. As a result, the advancement of knowledge in this field may be slowed down, underscoring the need for more open data-sharing practices and standardized evaluation frameworks.

Our contributions In this work, we introduce the first publicly accessible and open benchmark, comprising both a dataset and evaluation metrics, for developing chlorine state estimation methodologies for water distribution networks (WDNs). Additionally, we propose and assess two different deep neural networks – physics-guided Recurrent Neural Networks (RNNs) and physics-informed Graph Neural Networks (GNNs) – to serve as baselines for machine learning-based chlorine state estimation in the form of neural surrogate models.

Foundations

Water Quality Modeling

Water quality defines the suitability of water for consumption by humans and animals. It is determined by the concentration of specific key variables in the water. The dynamics of these concentrations describe how the mass per unit volume (concentration) of physical, chemical, and biological constituents (referred to as *species*) within the WDN vary across space and time.

Let $w \in \mathbb{R}^{n_w}$ be a vector indicating the concentration of n_w substances within a WDN at a certain location and time, with $w^{(i)}$ for $i \in \{1, \dots, n_w\}$ corresponding to the concentration of the ith species. We are typically concerned with a subset of species that must be regulated due to their health impact. For instance, $w^{(1)}$ may represent the concentration of a disinfectant (e.g., Chlorine), and $w^{(2)}$ may represent bacteria (e.g., *E. coli*).

In general, multiple species may coexist within a WDN. Some react to form new chemical compounds, while others are non-reactive and may remain constant. Both types are diluted and transferred, changing concentration at different network locations over time. This constitutes a dynamical system governed by advection and reaction dynamics.

In the WDN, when substances enter a pipe (edge $e_{(i,j)}$), they are transported with the water flow. This change in substance concentration over space and time is called *advection* and is typically described by a first-order hyperbolic partial differential equation:

$$\frac{\partial w_{(i,j)}(z,t)}{\partial t} + \frac{q_{(i,j)}(t)}{\alpha_{(i,j)}} \frac{\partial w_{(i,j)}(z,t)}{\partial z} = 0, \tag{1}$$

where $w_{(i,j)}(z,t)$ is the vector of substance concentrations at time t and distance z along pipe $e_{(i,j)}$ with flow $q_{(i,j)}(t)$ and cross-sectional area $\alpha_{(i,j)}$. Note that $\frac{q_{(i,j)}(t)}{\alpha_{(i,j)}}$ yields the water velocity in the respective pipe.

The reaction dynamics characterize how the concentration of one or more substances changes due to reactions, decay, or growth within a finite volume of water. These are typically described by single-species reaction dynamics, which model the rate of decay or growth for a single substance, aggregating the interactions with others.

For modeling multiple species, reaction dynamics in a static environment (i.e., neglecting the z dimension) involve coupled differential and algebraic equations:

$$\frac{dw(t)}{dt} = f_r(w(t), \Theta_r), \tag{2}$$

where $w \in \mathbb{R}^{n_w}$ is a vector of concentrations of n_w substances, $f_r(\cdot)$ describes the concentration change due to decay/growth reactions, and Θ_r represents reaction kinetics. In practice, f_r is derived from the stoichiometry of the reactions and may include linear or bilinear terms. A special case of (2) is the first-order decay model:

$$\frac{dw(t)}{dt} = \theta_r w(t), \tag{3}$$

where $\theta_r \in \Theta_r$ is a parameter vector indicating decay/growth rates for the substances in w.

Combining the advection and the reaction dynamics yields a PDE for water quality modeling that can be applied to a multi-species case:

$$\begin{aligned} \frac{\partial w_{(i,j)}(z,t)}{\partial t} + \frac{q_{(i,j)}(t)}{\alpha_{(i,j)}} \frac{\partial w_{(i,j)}(z,t)}{\partial z} &= f_r(w_{(i,j)}(z,t), \Theta_r) \\ &+ u^q_{(i,j)}(z,t) \\ &+ \phi_{(i,j)}(z,t), \end{aligned} \tag{4}$$

were $f_r(\cdot)$ denotes changes in concentration due to reactions, $u^q_{(i,j)}$ represents controlled inputs, such as added disinfectants, and $\phi_{(i,j)}(z,t)$ represents uncontrolled contamination inputs, such as accidental injections.

Water quality in tanks is often modeled using a Continuous Stirred-Tank Reactor (CSTR) approach [12], which assumes uniform mixing within the tank. For the ith tank $v_i \in \mathcal{V}_T$, the average concentration $x^q_i(k) = w_i(k)$ can be computed based on inflows, outflows, and reactions occurring within the tank. However, in this work, we omit the modeling of water quality within tanks as the primary focus is on the distribution network itself, where the dynamics of water quality and contaminant spread are more complex and directly relevant to end-user consumption.

Equation (4) represents physical constraints that can guide model predictions by incorporating known physical principles-such as chlorine decay and transport dynamics-ensuring that the learned models not only fit the data but also adhere to real-world water quality behaviors. Beyond replicating the system's dynamics, additional physical constraints on parameter values are incorporated into these models. These include enforcing a minimum chlorine concentration (which must be greater than or equal to zero) and ensuring that the maximum chlorine concentration does not exceed the injection point concentration. Such constraints enhance model robustness and prevent physically unrealistic predictions.

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a type of neural network architecture distinguished by their ability to incorporate memory components, enabling them to effectively process and analyze sequences of data points, such as time series. This memory capability allows RNNs to retain information from previous inputs, making them particularly well-suited for tasks where context or historical data is important. Unlike traditional neural networks, which treat each input independently, RNNs leverage their recurrent connections to maintain a dynamic internal state, which is updated as each new data point in the sequence is processed. This characteristic makes them highly valuable in applications like natural language processing, speech recognition, and any domain where understanding the temporal dependencies and patterns within sequential data is crucial. However, RNNs can face challenges such as vanishing or exploding gradients during training, which has led to the development of advanced variants like Long Short-Term Memory (LSTM) networks [13] and Gated Recurrent Units (GRUs) [14] to address these issues and enhance their performance.

The central concept of RNNs is their memory which is usually realized by introducing a state vector $\vec{h}_t \in \mathbb{R}^k$ that is updated over time (by a parametrized function) as the sequence of data points is processed:

$$(\vec{x}_t, \vec{h}_t) \mapsto (y_t, \vec{h}_{t+1})$$

The aforementioned popular instances of RNNs such as LSTMs [13] and GRUs [14] mainly differ in the way the hidden state $\vec{h}_t$ is updated and how the output y_t is computed.

Graph Neural Networks

Graph neural networks (GNNs) [15–17] are gaining increasing popularity for applications in WDNs. GNNs have been

successfully applied to hydraulic state estimation [18], surrogate modeling [19], and water quality estimation [7, 8]. One main motivator for this development is that WDNs are graph-structured domains. A WDN can be modeled as a graph by formalizing nodes (e.g. junctions, tanks, and reservoirs) as a set $V = \{v_1, .., v_N\}$, and pipes as edges $\mathcal{E} = \{e_{ij} : \forall i \in V; j \in \mathcal{N}(i)\}$. Here $\mathcal{N}(i)$ denotes the set of neighbors of node i. The edge e_{ij} is directed from the sender node i to the receiver node j. The nodes are further specified by feature vectors $X = [\mathbf{x}_1, ..., \mathbf{x}_N]$ to encode attributes like demand, node type, and other features. The edge feature vectors $Q = [\mathbf{q}_1, ..., \mathbf{q}_{|\mathcal{E}|}]$ typically contain information on pipe lengths, diameters, roughness coefficients, and flows.

A variety of different GNNs exist, and many of them apply *message-passing*, a principle that can be understood as three successive operations: Message generation, message aggregation, and node update. These operations are implemented by neural networks. The kth GNN layer computes the node embedding for node i via

$$\mathbf{x}_i^{(k)} = \gamma^{(k)}\left(\mathbf{x}_i^{(k-1)}, \sum_{j \in \mathcal{N}(i)} \phi^{(k)}\left(\mathbf{x}_i^{(k-1)}, \mathbf{x}_j^{(k-1)}, \mathbf{q}_{j,i}\right)\right),$$

where $\phi^{(k)}$ generates messages that are aggregated over the neighborhood $\mathcal{N}(i)$ by a permutation invariant aggregation function $\sum$, and $\gamma^{(k)}$ computes the output embedding of node i after the kth layer.

GNN for Physics-Informed Machine Learning

The quality of water in WDS follows complex spatio-temporal dynamics, and data is often limited, especially in real-world scenarios, where only a small number of sensors is available. This scarcity of data in such a complex system makes machine learning models prone to overfitting. To develop models that generalize effectively, incorporating constraints and regularization techniques is often beneficial. One promising approach to achieve this is through the use of physical and chemical prior knowledge, which is the focus of physics-informed machine learning.

The basic principle of physics-informed machine learning is to use prior knowledge like constraints, symmetries, and governing equations to regularize machine learning models [20, 21]. These governing equations typically exist in the form of ordinary differential equations (ODEs) and PDEs. Several works have discovered the close similarity between the operations in a GNN and PDEs and adapted the message-passing operations to resemble specific existing PDEs (i.e., heat diffusion, advection) [22–24]. These types of models have been called continuous GNNs [25]. It has also been highlighted that many numerical schemes to solve PDEs are special cases of message-passing [26] as well, which is the main operation of a GNN layer. Consequently, GNNs have already been applied to multiple physical problems where physical knowledge is used as inductive biases to the model [19, 21, 27, 28]. These findings motivate the use of a GNN for water quality estimation—a system governed by advection and reaction dynamics that arise from local interactions.

In this work, we introduce a simple baseline GNN that learns a PDE as described in Sect. 3.3.2. To estimate chlorine concentrations we apply Euler integration to generate time series of chlorine levels. This presents an alternative approach to other temporal graph neural networks like gated graph neural network (GGNN) that was used to estimate water quality in [7]. Primarily, while GGNN handles temporal dynamics explicitly via gating mechanisms, the PDE GNN handles them implicitly in the continuous evolution of the state. In future work, the PDE GNN can be extended with the existing research of continuous GNNs and physics-informed machine learning.

A Benchmark for Chlorine State Estimation

Currently, there do not exist any substantial benchmarks on the increasingly important problem of chlorine state estimation in water distribution networks (WDNs). We therefore propose an innovative benchmark designed to consistently evaluate and compare algorithms/methods for chlorine state estimation in WDNs by offering evaluation functions and comparisons. Furthermore, it also incorporates a framework to generate reasonable benchmarks such that researchers can easily create their own, modified, version of this proposed benchmark.

This benchmark offers considerable flexibility and can be applied to different tasks and configurations, including:

- Predicting chlorine concentration at a single node or at multiple nodes.
- Assuming full knowledge of all flows/chlorine concentrations, or only partial noisy knowledge, with the latter mimicking real-world use cases.

The entire benchmark, including the Python code for running simulations and generating the data, is publicly available on GitHub[1] and on the WaterBenchmarkHub platform [29][2]. The benchmark includes a dataset, along with a set of evaluation metrics, which both can be easily utilized in Python through the provided Python module. Comprehensive

[1] https://github.com/andreartelt/A-Benchmark-for-Physics-informed-Machine-Learning-of-Chlorine-States-in-Water-Distribution-Networks.

[2] https://waterfutures.github.io/WaterBenchmarkHub/benchmarks/ClStateEstimate.html.

documentation accompanies both the benchmark and the Python modules to facilitate their use.

Data Set

This benchmark contains scenarios for three different widely recognized Water Distribution Networks (WDNs) from the literature used for benchmarking: Net1, Hanoi, and the more recent and complex CY-DBP.

For the Net1 and Hanoi network (refer to Fig. 1 for an illustration), we build on the LeakDB [30] benchmark – a benchmark dataset comprising numerous artificially generated yet realistic (leakage) scenarios, designed to facilitate the evaluation and comparison of leakage detection algorithms.

For the CY-DBP network [31] (see Fig. 2 for an illustration), we mimic the uncertainties from LeakDB [30] to generate a large number of similar but slightly different scenarios.

The benchmark dataset in this work focuses on the transport, decay, and mixing of chlorine within water distribution networks, without explicitly modeling other substances. Its primary use case is chlorine state estimation, meaning it is intended to predict chlorine concentration at specific nodes or links in the network. The dataset is crafted to assess how effectively the physical processes—i.e. transport, decay, and mixing—are represented in a learned model. It includes scenarios designed for assessing adherence to underlying physical principles, such as those involving random demand patterns, which are artificially generated and hence easily scalable. The data set was created and simulated using the open-source Python package EPyT-Flow [32], which is based on EPyT [33] and EPANET [34]. As the quality dynamics depend on hydraulic dynamics [6], the hydraulics (e.g., flow rates, pressures, etc.) are calculated first, followed by the simulation of the quality dynamics.

For each of the three WDNs (i.e. Net1, Hanoi, and CY-DBP) the proposed benchmark includes 1000 scenarios, each lasting 30 days with 30-min time steps. These scenarios incorporate slightly varied demand patterns and network parameters, such as pipe lengths and diameters, to simulate uncertainties and evaluate the generalizability of the applied methods.

In all scenarios, a chlorine injection pump is placed at the network's water inlet (reservoir), with each scenario featuring a separate instance for one of three distinct injection patterns illustrated in Fig. 3. Note that the CY-DBP network has two reservoirs, and therefore two chlorine injection pumps. Also, both the Net1 and the CY-DBP network contain tanks that introduce additional complexity in the dynamics. While the chlorine injection scenarios considered here are not representative of real-world practices, they enable empirical evaluation of adherence to physical principles. Successfully predicting chlorine states under these conditions demonstrates that the processes of transport, mixing, and decay have been effectively learned or



Fig. 2 The CY-DBP water distribution network: 284 junctions, 357 pipes, 2 pumps, 4 valves, 2 reservoirs, and 1 tank. The two pumps are connected in parallel to the same nodes, the plot therefore shows only a single pump

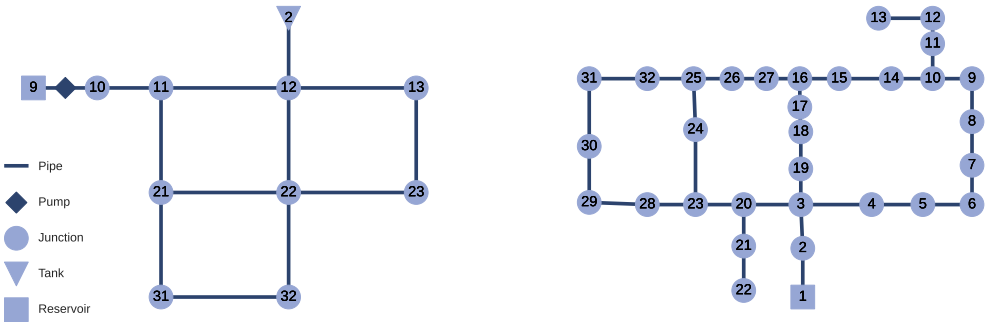


Fig. 1 Two popular water distribution networks. Left: Net1; Right: Hanoi

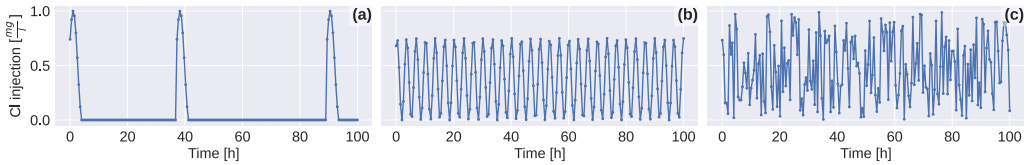


Fig. 3 Different Chlorine injection patterns: **a** Spike pattern, **b** periodic Wave-like pattern, **c** completely random pattern

modeled, as there are no extraneous correlations to rely upon. The injection patterns are designed as follows:

- **Spike**: Short bursts (i.e., spikes) of chlorine, as shown in Fig. 3a. The time intervals between impulses are randomized but always exceed the maximum transport delay (i.e., the time it takes chlorine to reach the most distant node). This pattern is particularly useful for understanding and learning substance transport times within the network, emphasizing advection dynamics.
- **Wave**: A periodic, wave-like pattern with a fixed but random frequency, illustrated in Fig. 3b. This pattern aids in learning chlorine decay rates, focusing on reaction dynamics.
- **Random**: A completely random pattern, depicted in Fig. 3c. This pattern is valuable for assessing both advection and reaction dynamics, offering a comprehensive test of the physical processes.

Furthermore, we also provide a version of each scenario where the demands have been randomized – i.e. breaking any potential correlations between demands and chlorine states.

The dataset comprises a total of 18,000 scenarios, providing information on the flow rate at each link, as well as the chlorine concentration at each node and link. Users have the flexibility to select only a subset of these quantities as input, simulating a specific sensor configuration. In this work however, we aim to replicate the simulator's behavior by assuming full knowledge of the hydraulics, including flow rates, and the chlorine concentration only at the injection points (i.e., reservoirs). Our objective is to substitute the quality simulation with a neural surrogate model implemented by a deep neural network.

Usage in Python To enhance accessibility, a Python module for loading the dataset is also provided. This includes a general data loader and a specialized loader for use with PyTorch [35].

```
# Load all 1000 Hanoi scenarios with the spike injection pattern
# Goal: Predict Cl concentration at node "2"
d = DataLoader()
X, y = d.load_data_from_scenarios(net_desc="Hanoi",
                                  random_demands=False,
                                  cl_injection_pattern_desc="spike",
                                  target_node_id="2")

# Load the topology of the Hanoi network
hanoi_topology = d.load_network_topology(net_desc="Hanoi")
```

Evaluation Metrics

We propose assessing performance using various, carefully chosen metrics. Here, we aim for easy-to-interpret metrics as well as metrics that are well-suited for the specific characteristics of the proposed benchmark. In particular, in the spike pattern (see Fig. 3a) where the ground truth chlorine concentration is zero most times, metrics such as the MAPE and R^2 would suffer from numerical issues (i.e. division by almost zero).

For a single node prediction, denoted as $\hat{y}_i$, this can be extended to multiple nodes by, for example, averaging over all nodes i:

- **Non-negativity of the predicted chlorine concentrations**—i.e. evaluating a trivial physical plausibility of the predicted concentrations:

$$\sum_{t=1}^{T} \mathbb{1}(\hat{y}_i(t) \geq 0) \tag{5}$$

- **Upper bound of a physically plausible chlorine concentration.** This depends on the maximum concentration at the injection points during past time steps, which may still influence the system. This concept is referred to as the memory of the system and relates to the maximum time required for water from the injection location to reach any node in the network. Given that flow rates are assumed to be known in this study, the maximum transport times for all nodes can be explicitly calculated. This also relates to the accuracy of predictions for each node, as a longer transport time implies a greater range of past inputs that can affect the output, introducing more uncertainty into the model.

$$\frac{1}{T-K}\sum_{t=K}^{T}\mathbb{1}\left(\hat{y}_i(t) \leq \max_{k\in[t-K,t]}(y_r(k))\right) \tag{6}$$

where K refers to the maximum transport time in the water network, and y_r refers to the chlorine concentration at the injection location over time—note that this generalizes for multiple injection sources.

- **The Mean-absolute-error (MAE)** as a standardized and easy-to-interpret error metrics:

$$\frac{1}{T}\sum_{t=1}^{T}\left|\hat{y}_i(t) - y_i(t)\right| \tag{7}$$

where T refers to the length of the time horizon—i.e. length of the simulated scenario.

- **The running MAE** for evaluating the performance over time by accumulating the performance up to some given time point. By this, we evaluate whether the performance remains stable and robust throughout the entire duration. The running MAE is a function that maps a time horizon $k \leq T$ to accumulated performance (i.e. MAE):

$$f(k) = \frac{1}{k}\sum_{t=1}^{k}\left|\hat{y}_i(t) - y_i(t)\right| \tag{8}$$

- **The amount of chlorine concentration which the model is over/underestimating**—i.e. indicating a bias for over or undershooting the true concentration. For this purpose, we sum up all positive and negative errors, and take their difference. A result close to zero indicates no bias, whereas a positive/negative result indicates a bias for over/underestimating the true concentration:

$$\left(\sum_{t=1}^{T}\max(0, e_i(t))\right) - \left(\sum_{t=1}^{T}\max(0, -1\cdot e_i(t))\right) \tag{9}$$

where

$$e_i(t) = \hat{y}_i(t) - y_i(t) \tag{10}$$

In addition to averaging scores across all nodes, we recommend comparing scores for each node individually. This approach allows us to assess whether errors and performance are uniformly distributed throughout the WDN or if certain nodes pose greater challenges than others. For example, a method's performance might be influenced by the node's distance from the reservoir (chlorine injection point)— i.e. predicting concentrations at nodes located farther away could be more challenging than at nodes closer to the injection site. One could also group nodes according to their transport delay—i.e. the time (e.g. time steps) it takes for a substance to travel from the injection point to the node of interest. Note that the transport delay might differ from the spatial distance (e.g. shortest path).

Usage in Python All the previously mentioned evaluation metrics are already implemented within the Python module included in the provided benchmark.

Baseline Neural Surrogate Models

Here, we consider the problem of predicting chlorine states at all nodes in the network based on the chlorine injection at the reservoir and the flows at all pipes. This problem is equivalent to the simulation of the chlorine dynamics based on some given hydraulic simulation results like those performed in EPANET [34]. However, we aim for a more efficient and differentiable solution through surrogate models. Consequently, we introduce two baseline models for developing such a surrogate model within our proposed benchmark. Besides the dynamics such as mixing and decay, the major challenge in this context is accounting for transport delays—i.e. the time it takes for chlorine to reach specific nodes. This can be tackled using models with memory components, such as recurrent neural networks, or by employing models that locally integrate dynamics, like graph neural networks, which inherently create a memory component. We therefore implement and evaluate a recurrent neural network, and graph neural networks as baselines for chlorine state estimation. These neural networks can serve as state-of-the-art baselines for future research in machine learning-based chlorine state estimation. These models serve as examples of a *physics-guided* neural network and a *physics-encoded* neural network, respectively. These terms are part of a taxonomy proposed in [36] that describes models that derive physical relations purely from the data as *physics-guided* and models that directly adopt the structure of a known physical formula into the architecture as *physics-encoded*.

The Python implementation of the experiments is also included in the GitHub Repository of the benchmark[3].

```
# Load test data
X_test, y_test = ....

# Predict Cl states
y_test_pred = ....

# Evaluate predictions using all proposed metrics
print(Evaluator.evaluate_predictions(y_test_pred, y_test))
```

[3] https://github.com/andreartelt/A-Benchmark-for-Physics-informed-Machine-Learning-of-Chlorine-States-in-Water-Distribution-Networks.

Recurrent Neural Network for Single Node Chlorine State Estimation

Our proposed deep recurrent neural network (RNN) for estimating chlorine concentration at a single node features two hidden Long Short-Term Memory (LSTM) [13] layers, with 128 and 64 neurons respectively, using tanh activation functions. A final dense layer with ReLU activation ensures non-negative predictions, as concentrations cannot be negative. The RNN takes as input the flow rate over time for every link in the WDN, as well as the chlorine concentration over time at the reservoir (injection point). Its task is to predict the chlorine concentration over time at a specified node in the WDN. The RNN is trained using the Adam optimizer [37] for up to 500 epochs, aiming to minimize the mean-squared error. Early stopping is implemented, halting the training if no improvement is observed over ten consecutive epochs. The dimensions of the hidden layers were optimized by a grid search (1–4 hidden layers, dimensions in [32, 64, 128, 256]) with a preference for lower values on the Net1 scenarios only, and recycled for all other networks due to the computational limitations of running a grid search for the larger water networks. Inputs to the RNN are standardized to prevent numerical issues during training. More details can be found in the accompanied Python code.

Because the data in our proposed benchmark data set (Sect. 3.1) is carefully crafted to model different dynamics such as transport delay, as well as breaking any correlations by random demand patterns and random chlorine injections, any model that is trained on it becomes a *physics-guided* model. Consequently, we refer to our proposed RNN-based surrogate model as a *physics-guided surrogate model.*

Graph Neural Network for Chlorine State Estimation

We now detail our GNN model for chlorine state estimation. We employ a message-passing neural network (MPNN) [17] to act as a PDE function and to model the dynamical system of chlorine levels. To do this, we first define the problem as an initial value problem with boundary conditions. The boundary conditions are chlorine injections at the reservoir and the initial state $\mathbf{w}(0)$ is the first sample of a time series of the chlorine concentration data. The chlorine injections constitute a Dirichlet boundary condition $w_{\text{Reservoir}}(t) = w_{\text{Injection}}(t)$, because the chlorine concentration at the reservoir node is known. As described in Sect. 2.1, the governing equations in water quality are the advection and reaction PDEs. These PDEs are typically solved at a much smaller time scale and a high-resolution domain (the pipes of the WDN are subdivided into small segments). Instead of learning these known PDEs, we aim to approximate the unknown dynamical system that generates our measured data directly. I.e. we model chlorine distributions on the WDN graph at a coarser temporal resolution directly. This raises the issue that the chlorine concentrations $\mathbf{w}(t)$ at time t do not fully determine $\frac{\partial \mathbf{w}(t)}{\partial t}$, as chlorine distributed along the pipes is not contained in $\mathbf{w}$. We address this by learning the PDE in a d-dimensional latent space $\mathbf{h} \in \mathbb{R}^{N \times d}$, similar to the hidden state of an RNN. Consequently, we first project the initial condition $\mathbf{w}(0)$ into the latent space, apply the GNN to integrate the initial state using Euler integration for T time steps and finally project the resulting outputs $H = [\mathbf{h}(0), .., \mathbf{h}(T)]$ back into the data space.

$$\mathbf{h}(0) = h_\phi(\mathbf{w}(0)) \tag{11}$$

$$\mathbf{h}(t + \delta t) = \mathbf{h}(t) + \delta t \cdot f_\theta(\mathbf{h}(t), \mathbf{q}(t)) \tag{12}$$

$$\mathbf{w}(t) = k_\psi(\mathbf{h}(t)) \tag{13}$$

where h and k are linear layers, f is a GNN that models the temporal change of chlorine concentrations, $\mathbf{q}(t) \in \mathbb{R}^{|\mathcal{E}| \times 2}$ are normalized flow velocities and pipe lengths that we use as edge features, and δt is the step size used for the Euler integration steps, we use $\delta t = \frac{1}{3}$ which corresponds to 10 minutes. Model parameters are denoted by ϕ, θ and ψ. The GNN models

$$f_\theta(\mathbf{h}(t), \mathbf{q}(t)) = \frac{\partial \mathbf{h}(t)}{\partial t}$$

and computes residuals for a node i following the general formula of a GNN layer as given in Sect. 2.3 by

$$\frac{\partial \mathbf{h}_i(t)}{\partial t} = \alpha \left(\sum_{j \in \mathcal{N}(i)} \phi\big((\mathbf{h}_j(t) - \mathbf{h}_i(t)) \parallel \mathbf{q}_{j,i}\big) - \mathbf{h}_i(t) \right), \tag{14}$$

where α is a learnable scalar, ϕ is a ReLU-activated 2-layer MLP and $\cdot \parallel \cdot$ denotes vector concatenation. Note that because the GNN incorporates message-passing, spatial relationships can be modeled allowing f_θ to function as a PDE. The GNN is trained on sequences of length 30 time steps as we experienced stability problems when using longer training sequences. We use the Adam optimizer for 300 epochs minimizing the mean absolute error using a batch size of 32. We apply early stopping with a patience of 15 epochs and a learning rate schedule that reduces the learning rate by a

factor of 0.5 when the validation loss does not decrease over 12 epochs. The chlorine values are normalized between 0 and 1, i.e. we divide by the maximal chlorine concentration observed at the injection nodes in the training split.

Experiments

Setup

We consider five different training configurations:

- Scenarios with normal demand patterns:
  - Spike chlorine injection patterns
  - Spike and random chlorine injection patterns
- Scenarios with randomized demand patterns:
  - Spike chlorine injection patterns
  - Spike and random chlorine injection patterns
- Scenarios with normal and randomized demand patterns: Spike and random chlorine injection patterns.

In every training setup, we randomly divide the scenarios into training (70%), validation (10%), and testing (20%) groups. If a configuration, such as a wave injection pattern, is not included in the training setup, we utilize all the scenarios of that configuration for evaluation.

We assess the performance of each injection pattern separately under both normal and randomized demand conditions using the proposed evaluation metrics (refer to "Evaluation Metrics"). A selection of the results (on the test set) is presented in "Analysis of Results", with additional details provided in Appendix A. Our focus is on the "interesting" cases where our proposed baseline neural surrogate models do not perform well, highlighting challenging scenarios within the benchmark.

One such challenge is the presence of a tank in the *CY-DBP* network, separating it into two sub-networks (s. Figure 2). Tanks exhibit varying dynamics depending on their water level and can drastically smooth out the incoming signal. This prevents the application of a simple PDE-inspired GNN, as the local function that is learned by such a model cannot handle the discontinuity that the tank introduces. For this reason, we only benchmark the RNN on *CY-DBP* and leave the application of a GNN for future work.

Computational Resources All computations were performed on a GPU cluster whereby a single GPU (NVIDIA A40) was used for training a model. In the case of the RNNs (i.e. single node prediction), the training took (depending on early stopping) up to approx. 2 h on the CY-DBP network, and approx. 40 min in the case of Net1. All together, thanks to the cluster parallelization, training and evaluation of all RNN baselines runs within approx. 48 hours. While

Table 1 Performance (*MAE*) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$0.01_{\pm 0.01}$	$0.25_{\pm 0.09}$	$0.22_{\pm 0.07}$	$0.01_{\pm 0.01}$	$0.23_{\pm 0.09}$	$0.2_{\pm 0.07}$
		Spike + Rand	$0.01_{\pm 0.01}$	$0.11_{\pm 0.05}$	$0.11_{\pm 0.04}$	$0.01_{\pm 0.01}$	$0.12_{\pm 0.05}$	$0.12_{\pm 0.04}$
	Rand	Spike	$0.02_{\pm 0.01}$	$0.24_{\pm 0.09}$	$0.24_{\pm 0.08}$	$0.01_{\pm 0.01}$	$0.22_{\pm 0.1}$	$0.2_{\pm 0.09}$
		Spike + Rand	$0.03_{\pm 0.02}$	$0.16_{\pm 0.05}$	$0.14_{\pm 0.04}$	$0.01_{\pm 0.01}$	$0.12_{\pm 0.04}$	$0.11_{\pm 0.04}$
	All	Spike + Rand	$0.01_{\pm 0.0}$	$0.11_{\pm 0.04}$	$0.1_{\pm 0.03}$	$0.01_{\pm 0.0}$	$0.11_{\pm 0.04}$	$0.1_{\pm 0.04}$

Table 2 Performance (*MAE*) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$0.05_{\pm 0.01}$	$0.39_{\pm 0.22}$	$0.41_{\pm 0.04}$	$0.06_{\pm 0.01}$	$0.41_{\pm 0.22}$	$0.42_{\pm 0.04}$
		Spike + Rand	$0.06_{\pm 0.04}$	$0.18_{\pm 0.13}$	$0.17_{\pm 0.05}$	$0.06_{\pm 0.05}$	$0.16_{\pm 0.13}$	$0.15_{\pm 0.07}$
	Rand	Spike	$0.05_{\pm 0.01}$	$0.4_{\pm 0.22}$	$0.43_{\pm 0.03}$	$0.05_{\pm 0.01}$	$0.42_{\pm 0.22}$	$0.44_{\pm 0.02}$
		Spike + Rand	$0.03_{\pm 0.01}$	$0.16_{\pm 0.13}$	$0.15_{\pm 0.05}$	$0.02_{\pm 0.01}$	$0.13_{\pm 0.12}$	$0.11_{\pm 0.06}$
	All	Spike + Rand	$0.04_{\pm 0.03}$	$0.16_{\pm 0.12}$	$0.15_{\pm 0.05}$	$0.04_{\pm 0.04}$	$0.13_{\pm 0.12}$	$0.11_{\pm 0.06}$

the entire benchmark is approx. 77.5 GB large, only a subset of those is required when training/evaluating a particular baseline model. For training an RNN baseline, the memory requirements are very modest. For instance, in the case of the CY-DBP network, a maximum of 350 MB on the GPU memory is required when training an RNN baseline for single-node prediction. The GNN has a slightly larger GPU memory requirement, as training is performed not on individual nodes but on the whole network. Nevertheless, training of the GNN on Net1 also takes approx. 40 min, this is mainly because it is trained on the first 30 time steps of each scenario and thus uses less data during training.

Analysis of Results

General Tables 1 and 2 show the performance (MAE Eq. (7) of the RNN and the GNN baseline surrogate models respectively. It can be observed that the RNN generally performs better than the GNN. However, while comparing these two surrogate models one should keep in mind that the RNN predicts single nodes only as we have separate models for each node. Instead, the GNN predicts all nodes simultaneously.

Comparing performances between different training configurations shows that the RNN is not as affected by changing from normal to randomized demand patterns as the GNN is. However, the results indicate that this is not an issue of generalization as both models show consistent transfer learning capabilities w.r.t. the demand pattern. Finally, including random chlorine injection patterns in the training generally shows a positive influence. Models trained on random injection patterns also show the best overall performance, especially for the GNN model. We include evaluations on the Net1 network and tables showing the other metrics in the Appendix (Tables 3, 4, 5, 6, 7, 8, 9 and 10), which support these findings.

Physically plausible concentration estimates Recall that our proposed benchmark comes with two metrics checking for severe violations of physics—i.e. non-negativity of concentrations and an upper bound based on the transport delay.

Note that by design both RNN and GNN can not output negative numbers—i.e. fulfilling the non-negativity constraint of concentrations. When it comes to the upper bound (based on the transport delay) as proposed in Eq. (6, we observe (see Tables 4, 9, 15) that the RNN almost always satisfies this upper bound, predicting physically plausible chlorine concentrations. This is quite remarkable given the fact that it does not contain any explicit physics constraints. All the physics constraints are implicitly in the carefully created training data, demonstrating the benefit of this *physics-guided* approach. We the exception of the spike injection pattern, the same observation can be made for the GNN (see Tables 6, 12)

Bias for over or underestimating From the results (Tables 3, 8, 14) we see that almost always the RNN has a bias towards underestimating the true chlorine concentration if it makes an error. This is a relevant finding since such a bias (positive or negative) might propagate in downstream tasks such as control. We also see that the GNN (Tables 5, 11) shows an even greater tendency to underestimate chlorine. To this end, both models show significantly lower bias when random injection patterns are included in the training set.

Performance over time and space In Fig. 4, we show the running MAE Eq. (8) of the RNN on the Hanoi network. Our observations indicate that: (1) The performance remains stable over time. (2) Training on randomized injection patterns considerably enhances performance. (3) The RNN's performance can vary significantly across different nodes. Further analysis shows that nodes located farther from the chlorine injection point at the reservoir (indicated by the node with zero error in dark blue) present more challenges

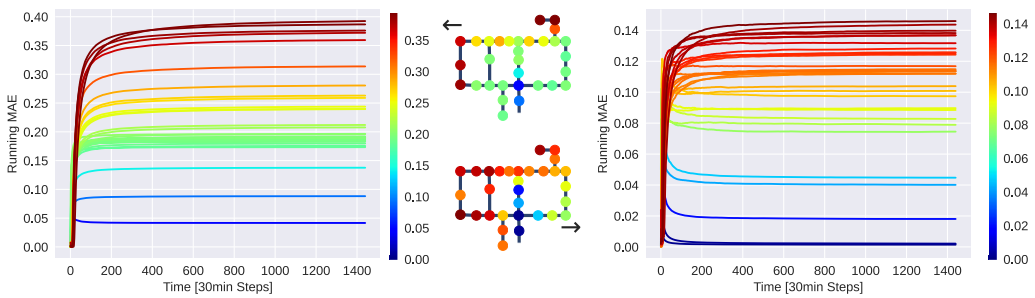


Fig. 4 Averaged running MAE of the *RNN* on the *Hanoi* network for randomized injections patterns and random demands (test set)—each colored line represents a different node. *Left:* Training was conducted using spike injection patterns. *Right:* Training involved both spike and random injection patterns. In both settings, the training was performed on randomized demand patterns. The colors show the total (average) MAE per node (last time step of the running MAE)

than nodes closer to the injection point. Similar performance variation can be observed in the GNN too, as indicated by Figs. 5 and 6, although, for the GNN, the performance does not seem to depend as much on the distance to the injection node. The observed variability can be attributed to several key factors. First, as chlorine travels through the network, it undergoes reaction processes that lead to gradual decay. Nodes farther from the injection point experience longer travel times, allowing for extended reaction durations, which increases uncertainty in concentration predictions. Second, it is well established in the scientific literature [6, 38] that variability in water flow is the greatest source of uncertainty in water quality models. The flow paths to distant nodes are often more complex, involving varying pipe diameters and multiple junctions where water mixing occurs. These factors introduce non-linearities in chlorine transport, making it more difficult for machine learning models to generalize.

To improve performance at challenging nodes, some alternative strategies can be explored, such as generating additional training data specifically targeting long-distance chlorine transport scenarios. Additionally, adjusting model loss functions to place greater emphasis on performance at distant nodes could help capture long-range dependencies more effectively.

Conclusion and Summary

In this work, we introduced a benchmark for chlorine state estimation in water distribution networks, featuring various chlorine injection patterns to capture diverse physical dynamics. This benchmark includes a comprehensive dataset comprising 18,000 scenarios and is supported by a Python module to facilitate easy access to the data and evaluation metrics within Python. Additionally, we introduce neural surrogate baselines using two different (physics-guided/informed) deep neural networks for chlorine state estimation and assess their performance using the proposed benchmark.

The empirical evaluation reveals that the RNN-based surrogate model typically outperforms the GNN-based surrogate model. When examining various training configurations, it appears that the RNN is less impacted by transitioning from standard to randomized demand patterns compared to the GNN. Nevertheless, the findings suggest that this is not a matter of generalization, as both models demonstrate consistent performance with respect to the demand pattern used in the evaluation dataset. Additionally, incorporating random chlorine injection patterns during training generally has a positive impact, with models trained on these patterns achieving the best overall performance. In terms of long-term robustness, both the RNN and GNN exhibit stable performance over time. However, there is a noticeable variation in performance across nodes, which depends significantly on their distance from the chlorine injection point. This suggests that substantial distances between the chlorine injection point and the target node present a significant challenge for the neural networks (also likely due to the longer time horizon).

Future work The proposed benchmark, along with the two baseline neural surrogate models, lays a foundation for future research in machine learning-based chlorine state estimation. In order to align with real-world constraints, future studies may move beyond the assumption of complete knowledge of all variables and explore configurations involving flow and chlorine sensors as well as uncertainties such as noisy measurements. Another interesting and highly important direction for future research would be to incorporate advanced reactions, such as additional species like contaminants, as well as time-varying and uncertain reactions.

Building on the presented baseline models, a promising direction for future research is refining the current PDE-inspired GNN. Our model is based on advection dynamics, which may not fully align with the partial observability of chlorine and the coarse temporal resolution of the proposed datasets. This mismatch adds complexity to the learning task and could be mitigated by further tailoring the PDE-based model to better account for these characteristics. Furthermore, other integration methods like Runge-Kutta, or implicit schemes could further improve the results.

Appendix A. Additional Empirical Results

Appendix A.1 Hanoi

See Tables 3, 4, 5, 6, and Figs. 5, 6

Table 3 Performance (over/underestimation Eq. (9) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	-3.42 ± 12.49	-208.14 ± 188.2	-72.82 ± 187.42	-3.71 ± 13.35	-186.38 ± 179.13	-29.91 ± 196.04
		Spike + Rand	-0.44 ± 1.01	-59.82 ± 23.11	2.39 ± 6.61	0.58 ± 3.2	-65.78 ± 24.66	-4.51 ± 9.11
	Rand	Spike	-11.93 ± 14.54	-280.17 ± 159.83	-220.21 ± 152.74	-1.05 ± 1.07	-276.6 ± 165.7	-201.04 ± 179.97
		Spike + Rand	18.81 ± 24.98	-70.54 ± 31.83	-14.47 ± 28.21	-0.37 ± 1.01	-64.96 ± 25.14	0.01 ± 6.26
	All	Spike + Rand	-0.77 ± 1.04	-65.81 ± 23.69	-2.04 ± 7.75	-0.43 ± 1.05	-66.74 ± 26.3	-0.66 ± 8.52

Table 4 Performance (Satisfaction of an upper bound satisfaction based on transport delay Eq. (6) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	1.0 ± 0.01	0.98 ± 0.02	1.0 ± 0.01	0.99 ± 0.01	0.98 ± 0.02	1.0 ± 0.0
		Spike + Rand	0.99 ± 0.01	1.0 ± 0.01	1.0 ± 0.01	0.98 ± 0.04	0.99 ± 0.01	1.0 ± 0.0
	Rand	Spike	0.97 ± 0.04	0.98 ± 0.01	1.0 ± 0.0	1.0 ± 0.01	0.99 ± 0.01	1.0 ± 0.0
		Spike + Rand	0.9 ± 0.06	0.97 ± 0.02	1.0 ± 0.01	0.99 ± 0.01	1.0 ± 0.01	1.0 ± 0.01
	All	Spike + Rand	1.0 ± 0.01	1.0 ± 0.01	1.0 ± 0.01	0.99 ± 0.01	1.0 ± 0.01	1.0 ± 0.01

Table 5 Performance (Over/underestimation) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	-51.13 ± 13.69	-482.43 ± 312.18	-579.1 ± 78.7	-57.35 ± 12.3	-532.44 ± 319.28	-598.18 ± 79.58
		Spike + Rand	37.14 ± 69.83	-123.72 ± 154.99	-81.62 ± 93.23	45.05 ± 78.16	-119.45 ± 142.48	-55.2 ± 81.9
	Rand	Spike	-68.15 ± 12.75	-556.26 ± 307.07	-616.86 ± 44.1	-72.05 ± 10.86	-591.8 ± 308.55	-635.05 ± 38.85
		Spike + Rand	-8.68 ± 10.23	-111.61 ± 140.63	-22.75 ± 43.13	-9.58 ± 7.16	-111.7 ± 135.03	-10.14 ± 18.51
	All	Spike + Rand	31.03 ± 46.84	-108.83 ± 153.45	-31.23 ± 52.67	32.65 ± 47.16	-95.62 ± 144.05	-3.54 ± 19.16

Table 6 Performance (Satisfaction of an upper bound based on transport delay Eq. (6) on the test set of differently trained models – we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	0.69 ± 0.46	1.0 ± 0.06	1.0 ± 0.0	0.73 ± 0.44	1.0 ± 0.02	1.0 ± 0.0
		Spike + Rand	0.56 ± 0.5	0.95 ± 0.22	1.0 ± 0.05	0.54 ± 0.5	0.98 ± 0.13	1.0 ± 0.06
	Rand	Spike	0.71 ± 0.45	1.0 ± 0.06	1.0 ± 0.0	0.73 ± 0.45	1.0 ± 0.02	1.0 ± 0.0
		Spike + Rand	0.71 ± 0.45	0.98 ± 0.14	1.0 ± 0.05	0.73 ± 0.44	0.99 ± 0.08	1.0 ± 0.03
	All	Spike + Rand	0.56 ± 0.5	0.96 ± 0.21	1.0 ± 0.05	0.53 ± 0.5	0.97 ± 0.16	1.0 ± 0.03

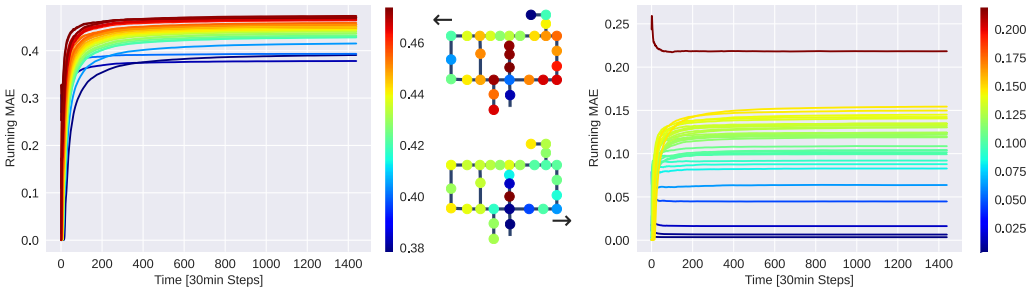


Fig. 5 Averaged running MAE of the GNN on the Hanoi network for randomized injections patterns and random demands (test set)—each colored line represents a different node. *Left:* Training was conducted using spike injection patterns. *Right:* Training involved both spike and random injection patterns. In both settings, the training was performed on randomized demand patterns. The colors show the total (average) MAE per node (last time step of the running MAE)



Fig. 6 GNN chlorine predictions vs. ground truth of different nodes in the Hanoi network. The GNN was trained on all demand patterns and *spike+random* injection patterns. Visualized are results on the test set with *random* demands and the *random* injection pattern

Appendix A.2. Net1

See Tables 7, 8, 9, 10, 11 and 12.

Table 7 Performance (MAE) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$0.02_{\pm 0.01}$	$0.1_{\pm 0.04}$	$0.09_{\pm 0.04}$	$0.02_{\pm 0.01}$	$0.1_{\pm 0.04}$	$0.1_{\pm 0.05}$
		Spike + Rand	$0.02_{\pm 0.01}$	$0.08_{\pm 0.03}$	$0.08_{\pm 0.04}$	$0.02_{\pm 0.01}$	$0.08_{\pm 0.03}$	$0.08_{\pm 0.04}$
	Rand	Spike	$0.02_{\pm 0.01}$	$0.11_{\pm 0.04}$	$0.1_{\pm 0.04}$	$0.02_{\pm 0.01}$	$0.1_{\pm 0.04}$	$0.1_{\pm 0.05}$
		Spike + Rand	$0.02_{\pm 0.01}$	$0.08_{\pm 0.03}$	$0.08_{\pm 0.04}$	$0.02_{\pm 0.01}$	$0.08_{\pm 0.03}$	$0.08_{\pm 0.04}$
	All	Spike + Rand	$0.02_{\pm 0.01}$	$0.06_{\pm 0.03}$	$0.07_{\pm 0.03}$	$0.02_{\pm 0.01}$	$0.06_{\pm 0.03}$	$0.08_{\pm 0.04}$

Table 8 Performance (over/underestimation) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$-1.22_{\pm 1.4}$	$-17.57_{\pm 21.1}$	$-2.38_{\pm 11.46}$	$-2.25_{\pm 3.11}$	$-21.6_{\pm 24.79}$	$-2.37_{\pm 15.61}$
		Spike + Rand	$-0.99_{\pm 1.4}$	$-19.3_{\pm 7.84}$	$0.52_{\pm 3.09}$	$1.77_{\pm 2.28}$	$-19.65_{\pm 8.73}$	$0.65_{\pm 4.14}$
	Rand	Spike	$-1.2_{\pm 4.04}$	$-21.04_{\pm 17.72}$	$-5.17_{\pm 9.56}$	$-0.76_{\pm 1.66}$	$-19.56_{\pm 20.6}$	$-1.16_{\pm 10.32}$
		Spike + Rand	$0.15_{\pm 4.92}$	$-21.16_{\pm 9.4}$	$-0.53_{\pm 6.9}$	$-2.5_{\pm 2.73}$	$-21.4_{\pm 8.94}$	$-0.43_{\pm 4.83}$
	All	Spike + Rand	$-0.63_{\pm 2.54}$	$-18.93_{\pm 7.73}$	$-1.28_{\pm 4.16}$	$-0.67_{\pm 2.44}$	$-20.21_{\pm 7.73}$	$-0.55_{\pm 4.64}$

Table 9 Performance (satisfaction of an upper bound based on transport delay Eq. (6) on the test set of differently trained models – we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.01}$	$0.99_{\pm 0.01}$	$1.0_{\pm 0.0}$
		Spike + Rand	$0.99_{\pm 0.01}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.01}$	$1.0_{\pm 0.01}$	$1.0_{\pm 0.0}$
	Rand	Spike	$0.99_{\pm 0.01}$	$0.99_{\pm 0.01}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.01}$	$0.99_{\pm 0.01}$	$1.0_{\pm 0.0}$
		Spike + Rand	$0.99_{\pm 0.01}$	$0.99_{\pm 0.0}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.01}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$
	All	Spike + Rand	$0.99_{\pm 0.01}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.01}$	$1.0_{\pm 0.0}$	$1.0_{\pm 0.0}$

Table 10 Performance (MAE) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	$0.12_{\pm 0.04}$	$0.19_{\pm 0.11}$	$0.19_{\pm 0.03}$	$0.13_{\pm 0.05}$	$0.22_{\pm 0.12}$	$0.19_{\pm 0.03}$
		Spike + Rand	$0.06_{\pm 0.03}$	$0.15_{\pm 0.11}$	$0.14_{\pm 0.06}$	$0.07_{\pm 0.04}$	$0.17_{\pm 0.12}$	$0.15_{\pm 0.06}$
	Rand	Spike	$0.08_{\pm 0.04}$	$0.18_{\pm 0.11}$	$0.19_{\pm 0.03}$	$0.09_{\pm 0.05}$	$0.2_{\pm 0.12}$	$0.19_{\pm 0.03}$
		Spike + Rand	$0.04_{\pm 0.02}$	$0.11_{\pm 0.08}$	$0.11_{\pm 0.04}$	$0.04_{\pm 0.03}$	$0.13_{\pm 0.09}$	$0.11_{\pm 0.05}$
	All	Spike + Rand	$0.04_{\pm 0.02}$	$0.11_{\pm 0.09}$	$0.1_{\pm 0.04}$	$0.04_{\pm 0.02}$	$0.13_{\pm 0.1}$	$0.1_{\pm 0.05}$

Table 11 Performance (over/underestimation) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	9.26 ± 80.03	-141.5 ± 191.05	-164.28 ± 94.04	-9.68 ± 88.48	-172.07 ± 205.97	-169.11 ± 89.99
		Spike + Rand	-29.0 ± 54.87	-121.61 ± 161.23	-95.97 ± 125.22	-34.66 ± 62.42	-144.21 ± 170.83	-93.72 ± 122.57
	Rand	Spike	-41.36 ± 64.9	-147.55 ± 174.09	-168.69 ± 95.54	-50.23 ± 73.4	-169.43 ± 180.91	-161.38 ± 91.33
		Spike + Rand	-13.67 ± 21.08	-80.63 ± 81.92	-53.67 ± 54.65	-18.53 ± 30.75	-103.81 ± 97.62	-65.49 ± 67.9
	All	Spike + Rand	-15.44 ± 20.1	-100.65 ± 105.47	-49.2 ± 39.23	-20.05 ± 23.64	-124.71 ± 114.77	-51.79 ± 40.53

Table 12 Performance (satisfaction of an upper bound based on transport delay Eq. (6) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	0.98 ± 0.14	0.94 ± 0.24	1.0 ± 0.02	0.99 ± 0.1	0.95 ± 0.21	1.0 ± 0.02
		Spike + Rand	0.98 ± 0.14	0.98 ± 0.15	1.0 ± 0.07	0.99 ± 0.12	0.98 ± 0.13	0.99 ± 0.08
	Rand	Spike	0.98 ± 0.13	0.97 ± 0.17	1.0 ± 0.01	0.99 ± 0.09	0.98 ± 0.14	1.0 ± 0.01
		Spike + Rand	0.98 ± 0.14	0.99 ± 0.11	1.0 ± 0.02	0.99 ± 0.12	0.99 ± 0.08	1.0 ± 0.02
	All	Spike + Rand	0.98 ± 0.14	0.99 ± 0.11	1.0 ± 0.03	0.99 ± 0.11	0.99 ± 0.09	1.0 ± 0.03

Appendix A.3. CY-DBP

See Tables 13, 14 and 15.

Table 13 Performance (MAE) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	0.05 ± 0.01	0.37 ± 0.09	0.38 ± 0.09	0.05 ± 0.01	0.39 ± 0.1	0.39 ± 0.09
		Spike + Rand	0.02 ± 0.01	0.16 ± 0.1	0.07 ± 0.13	0.03 ± 0.03	0.18 ± 0.1	0.09 ± 0.13
	Rand	Spike	0.05 ± 0.01	0.37 ± 0.09	0.38 ± 0.09	0.05 ± 0.01	0.39 ± 0.1	0.39 ± 0.1
		Spike + Rand	0.02 ± 0.01	0.13 ± 0.06	0.05 ± 0.07	0.01 ± 0.01	0.14 ± 0.06	0.03 ± 0.07
	All	Spike + Rand	0.01 ± 0.01	0.14 ± 0.07	0.04 ± 0.09	0.01 ± 0.01	0.15 ± 0.07	0.04 ± 0.09

Table 14 Performance (over/underestimation Eq. (9) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	-69.5 ± 20.0	-534.0 ± 133.5	-550.12 ± 139.9	-71.88 ± 19.46	-561.43 ± 139.02	-556.91 ± 140.05
		Spike + Rand	-10.62 ± 25.47	-161.38 ± 156.64	-83.06 ± 194.99	-3.22 ± 54.5	-191.62 ± 159.55	-98.82 ± 193.78
	Rand	Spike	-71.36 ± 17.71	-537.3 ± 131.16	-554.1 ± 135.27	-73.48 ± 17.83	-564.56 ± 137.97	-560.67 ± 137.01
		Spike + Rand	-13.24 ± 16.03	-138.96 ± 78.33	-44.2 ± 98.1	-2.96 ± 13.77	-125.11 ± 78.86	-21.56 ± 95.82
	All	Spike + Rand	-4.23 ± 16.37	-116.59 ± 100.51	-33.7 ± 124.12	-4.46 ± 16.65	-131.75 ± 103.61	-34.18 ± 125.53

Table 15 Performance (satisfaction of an upper bound based on transport delay Eq. (6) on the test set of differently trained models—we report the average (over all nodes and scenarios) and standard deviation rounded to two decimal places

	Demands		Evaluation					
			Normal			Rand		
		Cl pattern(s)	Spike	Wave	Rand	Spike	Wave	Rand
Training	Normal	Spike	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
		Spike + Rand	1.0 ± 0.0	0.97 ± 0.02	1.0 ± 0.0	1.0 ± 0.0	0.97 ± 0.03	1.0 ± 0.0
	Rand	Spike	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
		Spike + Rand	1.0 ± 0.0	0.98 ± 0.01	1.0 ± 0.0	1.0 ± 0.0	0.97 ± 0.02	1.0 ± 0.0
	All	Spike + Rand	1.0 ± 0.0	0.96 ± 0.02	1.0 ± 0.0	1.0 ± 0.0	0.97 ± 0.02	1.0 ± 0.0

Author Contributions The first two authors contributed equally. Conceptualization: Luca Hermes, André Artelt, Stelios G. Vrachimis; Methodology: Luca Hermes, André Artelt; *Formal analysis and investigation:* Luca Hermes, André Artelt, Stelios G. Vrachimis; Writing—original draft preparation: Luca Hermes, André Artelt, Stelios G. Vrachimis; Writing—review and editing: Luca Hermes, André Artelt, Stelios G. Vrachimis; Funding acquisition: Barbara Hammer, Marios M. Polycarpou; Supervision: Barbara Hammer, Marios M. Polycarpou.

Funding Open Access funding enabled and organized by Projekt DEAL. This research was supported by the Ministry of Culture and Science NRW (Germany) as part of the Lamarr Fellow Network. We also gratefully acknowledge funding from the European Research Council (ERC) under the ERC Synergy Grant Water-Futures (Grant agreement No. 951424). Funding from the project "SAIL: SustAInable Life-cycle of Intelligent Socio-Technical Systems" (Grant ID NW21-059 A), which is funded by the program "Netzwerke 2021" of the Ministry of Culture and Science of the State of North Rhine Westphalia, Germany. This publication reflects the views of the authors only.

Data Availability All data is publicly available.

Declarations

Conflict of Interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

Research Involving Human and/or Animals Not applicable.

Informed Consent Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Eliades DG, Vrachimis SG, Moghaddam A, Tzortzis I, Polycarpou MM. Contamination event diagnosis in drinking water networks: a review. Annu Rev Control. 2023;55:420–41. https://doi.org/10.1016/J.ARCONTROL.2023.03.011.
2. Tinelli S, Juran I. Artificial intelligence-based monitoring system of water quality parameters for early detection of non-specific bio-contamination in water distribution systems. Water Supply. 2019;19:1785–92. https://doi.org/10.2166/WS.2019.057.
3. Dogo EM, Nwulu NI, Twala B, Aigbavboa C. A survey of machine learning methods applied to anomaly detection on drinking-water quality data. Urban Water J. 2019;16:235–48. https://doi.org/10.1080/1573062X.2019.1637002. https://www.tandfonline.com/doi/abs/10.1080/1573062X.2019.1637002
4. Perelman L, Arad J, Housh M, Ostfeld A. Event detection in water distribution systems from multivariate water quality time series.

Environ Sci Technol. 2012;46(15):8212–9. https://doi.org/10.1021/es3014024.
5. Zou XY, Lin YL, Xu B, Guo ZB, Xia SJ, Zhang TY, Wang AQ, Gao NY. A novel event detection model for water distribution systems based on data-driven estimation and support vector machine classification. Water Resour Manage. 2019;33:4569–81. https://doi.org/10.1007/S11269-019-02317-5/FIGURES/6. https://link.springer.com/article/10.1007/s11269-019-02317-5
6. Vrachimis SG, Eliades DG, Polycarpou MM. Calculating chlorine concentration bounds in water distribution networks: a backtracking uncertainty bounding approach. Water Resour Res. 2021;57:e2020WR028684. https://doi.org/10.1029/2020WR028684.
7. Li Z, Liu H, Zhang C, Fu G. Real-time water quality prediction in water distribution networks using graph neural networks with sparse monitoring data. Water Res. 2024;250: 121018. https://doi.org/10.1016/j.watres.2023.121018.
8. Salem AK, Taha AF, Abokifa AA. Graph neural networks-based dynamic water quality state estimation in water distribution networks. Eng Appl Artif Intell. 2024;138: 109426. https://doi.org/10.1016/j.engappai.2024.109426.
9. Ostfeld A, Salomons E, Ormsbee L, Uber JG, Bros CM, Kalungi P, Burd R, Zazula-Coetzee B, Belrain T, Kang D, Lansey K, Shen H, McBean E, Wu ZY, Walski T, Alvisi S, Franchini M, Johnson JP, Ghimire SR, Barkdoll BD, Koppel T, Vassiljev A, Kim JH, Chung G, Yoo DG, Diao K, Zhou Y, Li J, Liu Z, Chang K, Gao J, Qu S, Yuan Y, Prasad TD, Laucelli D, Lyroudia L, Kapelan Z, Savic D, Berardi L, Barbaro G, Giustolisi O, Asadzadeh M, Tolson BA, McKillop R. Battle of the water calibration networks. J Water Resour Plan Manag. 2012;138(5):523–32. https://doi.org/10.1061/(ASCE)WR.1943-5452.0000191.
10. Riyadh A, Zayat A, Chaaban A, Peleato NM. Improving chlorine residual predictions in water distribution systems using recurrent neural networks. Environ Sci Water Res Technol. 2024;10(10):2533–45. https://doi.org/10.1039/D4EW00329B. (**publisher: The Royal Society of Chemistry**).
11. Li Z, Liu H, Zhang C, Fu G. Generative adversarial networks for detecting contamination events in water distribution systems using multi-parameter, multi-site water quality monitoring. Environ Sci Ecotechnol. 2023;14: 100231. https://doi.org/10.1016/J.ESE.2022.100231.
12. Hua P, de Oliveira K, Cheung P, Gonçalves FV, Zhang J. Influences of model structure and calibration data size on predicting chlorine residuals in water storage tanks. Sci Total Environ. 2018;634:705–14. https://doi.org/10.1016/j.scitotenv.2018.03.364.
13. Hochreiter S. Long short-term memory. Neural Computation MIT-Press; 1997.
14. Cho K. Learning phrase representations using rnn encoder-decoder for statistical machine translation. ArXiv Preprint. 2014. arXiv:1406.1078
15. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G. The graph neural network model. IEEE Trans Neural Netw. 2009;20(1):61–80. https://doi.org/10.1109/TNN.2008.2005605.
16. Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. CoRR abs/1609.02907 (2016). arXiv:1609.02907.
17. Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE. Neural message passing for quantum chemistry (2017). arXiv:1704.01212.
18. Ashraf I, Hermes L, Artelt A, Hammer B. Spatial graph convolution neural networks for water distribution systems. In: International Symposium on Intelligent Data Analysis, Springer, 2023, pp. 29–41.
19. Ashraf I, Strotherm J, Hermes L, Hammer B. Physics-informed graph neural networks for water distribution systems. 2024. https://doi.org/10.1609/aaai.v38i20.30192.
20. Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys. 2019;378:686–707. https://doi.org/10.1016/j.jcp.2018.10.045.https://www.sciencedirect.com/science/article/pii/S0021999118307125
21. Xiang Z, Peng W, Yao W, Liu X, Zhang X. Solving spatiotemporal partial differential equations with Physics-informed Graph Neural Network. Appl Soft Comput. 2024;155:111437. https://doi.org/10.1016/j.asoc.2024.111437. https://linkinghub.elsevier.com/retrieve/pii/S1568494624002114
22. Chamberlain BP, Rowbottom J, Gorinova M, Webb S, Rossi E, Bronstein MM. GRAND: Graph Neural Diffusion. 2021. arXiv:2106.10934 [cs, stat]
23. Choi J, Hong S, Park N, Cho S-B. gread: graph neural reaction-diffusion networks, 2023. arXiv:2211.14208 [cs].
24. Eliasof M, Haber E, Treister E. ADR-GNN: advection-diffusion-reaction graph neural networks. 2023. arXiv:2307.16092 [cs].
25. Xhonneux L-PAC, Qu M, Tang J. Continuous Graph Neural Networks. 2020. arXiv:1912.00967 [cs, stat].
26. Brandstetter J, Worrall D, Welling M. Message Passing Neural PDE Solvers. 2022. arXiv:2202.03376 [cs, math].
27. Horie M, Mitsume N. Physics-embedded neural networks: graph neural PDE solvers with mixed boundary conditions. Adv Neural Inf Process Syst. 2022;35:23218–29. https://proceedings.neurips.cc/paper_files/paper/2022/hash/93476ae409ae3246e22a9d4b931f84ed-Abstract-Conference.html
28. Dalton D, Husmeier D, Gao H. Physics-informed graph neural network emulation of soft-tissue mechanics. Comput Methods Appl Mech Eng. 2023;417:116351. https://doi.org/10.1016/j.cma.2023.116351.https://www.sciencedirect.com/science/article/pii/S0045782523004759
29. Artelt A, Giese K, Vrachimis SG, Eliades DG, Polycarpou MM, Hammer B. WaterBenchmarkHub, 2024. https://github.com/WaterFutures/WaterBenchmarkHub.
30. Vrachimis SG, Kyriakou MS, Eliades DG, Polycarpou MM. LeakDB: a benchmark dataset for leakage diagnosis in water distribution networks description of benchmark. In: WDSA / CCWI Joint Conference Proceedings, Vol. 1, 2018. https://doi.org/10.5281/zenodo.1313116.
31. Pavlou P, Kyriakou M, Vrachimis SG, Eliades DG. A comprehensive virtual testbed for modeling disinfection byproduct formation in water distribution networks. Eng Proc. 2024;69(1). https://doi.org/10.3390/engproc2024069033.https://www.mdpi.com/2673-4591/69/1/33
32. Artelt A, Kyriakou MS, Vrachimis SG, Eliades DG, Hammer B, Polycarpou MM. EPyT-flow: a toolkit for generating water distribution network data (2024). https://doi.org/10.21105/joss.07104.https://github.com/WaterFutures/EPyT-Flow
33. Kyriakou MS, Demetriades M, Vrachimis SG, Eliades DG, Polycarpou MM. EPyT: an EPANET-python toolkit for smart water network simulations. J Open Source Softw. 2023;8(92):5947. https://doi.org/10.21105/joss.05947.
34. Rossman LA. EPANET 2: users manual, Tech. rep., US Environmental Protection Agency. Office of Research and Development. National Risk Management Research Laboratory (2000). https://nepis.epa.gov/Adobe/PDF/P1007WWU.pdf.
35. Ansel J, Yang E, He H, Gimelshein N, Jain A, Voznesensky M, Bao B, Bell P, Berard D, Burovski E, Chauhan G, Chourdia A, Constable W, Desmaison A, DeVito Z, Ellison E, Feng W, Gong J, Gschwind M, Hirsh B, Huang S, Kalambarkar K, Kirsch L, Lazos M, Lezcano M, Liang Y, Liang J, Lu Y, Luk C, Maher B,

Pan Y, Puhrsch C, Reso M, Saroufim M, Siraichi MY, Suk H, Suo M, Tillet P, Wang E, Wang X, Wen W, Zhang S, Zhao X, Zhou K, Zou R, Mathews A, Chanan G, Wu P, Chintala S. PyTorch 2: faster machine learning through dynamic python bytecode transformation and graph compilation. In: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), ACM, 2024. https://doi.org/10.1145/3620665.3640366.https://pytorch.org/assets/pytorch2-2.pdf.

36. Faroughi SA, Pawar NM, Fernandes C, Raissi M, Das S, Kalantari NK, Kourosh Mahjour S. Physics-guided, physics-informed, and physics-encoded neural networks and operators in scientific computing: fluid and solid mechanics. J Comput Inf Sci Eng. 2024;24(4): 040802. https://doi.org/10.1115/1.4064449.
37. Kingma DP, Ba J. Adam: a method for stochastic optimization. In: Bengio Y, LeCun Y (Eds.) 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015. arxiv:org/abs/1412.6980.
38. Pasha M, Lansey K. Effect of parameter uncertainty on water quality predictions in distribution systems-case study. J Hydroinf. 2010;12(1):1–21. https://doi.org/10.2166/hydro.2010.053.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.